

LogiCode: an LLM-Driven Framework for Logical Anomaly Detection

Yiheng Zhang, Yunkang Cao, *Graduate Student Member, IEEE*, Xiaohao Xu, Weiming Shen*, *Fellow, IEEE*,

Abstract—This paper presents LogiCode, a novel framework that leverages Large Language Models (LLMs) for identifying logical anomalies in industrial settings, moving beyond traditional focus on structural inconsistencies. By harnessing LLMs for logical reasoning, LogiCode autonomously generates Python codes to pinpoint anomalies such as incorrect component quantities or missing elements, marking a significant leap forward in anomaly detection technologies. A custom dataset “LOCO-Annotations” and a benchmark “LogiBench” are introduced to evaluate the LogiCode’s performance across various metrics including binary classification accuracy, code generation success rate, and precision in reasoning. Findings demonstrate LogiCode’s enhanced interpretability, significantly improving the accuracy of logical anomaly detection and offering detailed explanations for identified anomalies. This represents a notable shift towards more intelligent, LLM-driven approaches in industrial anomaly detection, promising substantial impacts on industry-specific applications.

Note to Practitioners—This work introduces LogiCode, an innovative system leveraging Large Language Models (LLMs) for logical anomaly detection in industrial settings, shifting the paradigm from traditional visual inspection methods. LogiCode autonomously generates Python codes for logical anomaly detection, enhancing interpretability and accuracy. Our novel approach, validated through the “LOCO-Annotations” dataset and LogiBench benchmark, demonstrates superior performance in identifying logical anomalies, a challenge often encountered in complex industrial components like assembly and packaging. LogiCode provides a significant advancement in addressing the nuanced requirements of detecting logical anomalies, offering a robust and interpretable solution to practitioners seeking to enhance quality control and reduce manual inspection efforts.

Index Terms—Logical Anomaly Detection, Large Language Models, Industrial Anomaly Detection, Dataset Annotation.

I. INTRODUCTION

ANOMALY detection in industrial scenarios is pivotal for ensuring the quality and reliability of products [1], [2], [3]. Traditionally, this domain has been dominated by methods focusing on structural defects, such as dents or scratches [4], detectable through vision-based techniques. However, these conventional methods often fall short when dealing with logical anomalies [5] – errors in high-level semantic logic, such as incorrect orders in combinations of normal components, as illustrated in Fig. 1. These logical anomalies widely exist in industrial settings, like assembly and packaging. Given the potential for severe functional consequences associated with logical anomalies, specific logical anomaly detection methods have been developed.

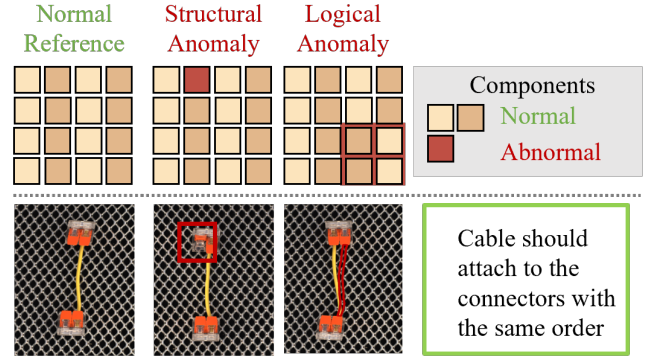


Fig. 1: Comparison between the structural anomaly and the logical anomaly. **(Top)** Toy example. **(Bottom)** Examples from the MVtec LOCO AD dataset [5]. In toy example, the yellow squares represent normal components, while the red squares signify structural deviations from the norm. Notably, even though the individual components in logical anomaly are normal, they collectively defy logical constraints. For the LOCO examples, structural anomaly is exemplified by damaged connector, while logical anomaly is indicated by misplaced cable connections.

Existing logical anomaly detection methods, such as GCAD [5] and its successors [6], [7], [8], have made their initial progresses, as shown in Fig. 2. These methods leverage both local and global network branches to detect structural and logical inconsistencies, with a particular emphasis on global consistency representations. They assume that global consistency representations can reflect the overall logical constraint of an image. However, such representations often fail to equate to the deep logical relationships necessary for identifying subtle logical anomalies—like an incorrect sequence of correctly quantified components or components that match in type but not in specification. The inherent challenge lies in detecting these high-level semantic inconsistencies, which requires an understanding of real logical reasoning and relationships beyond the capabilities of purely vision-based systems. Furthermore, these methods often present results for logical anomalies through anomaly scores (see Fig. 2). However, unlike structural anomaly detection, logical anomaly detection typically demands clearer explanations while the lack of explicit reasoning can lead to confusions. This highlights the need for a method that not only detects but also articulately explains logical anomalies.

Recent advances in LLMs [9], [10], [11], offer new insights and solutions for this challenge. LLMs excel in understanding and processing complex logical relationships and reasoning [12], [13], making them particularly suited for detecting

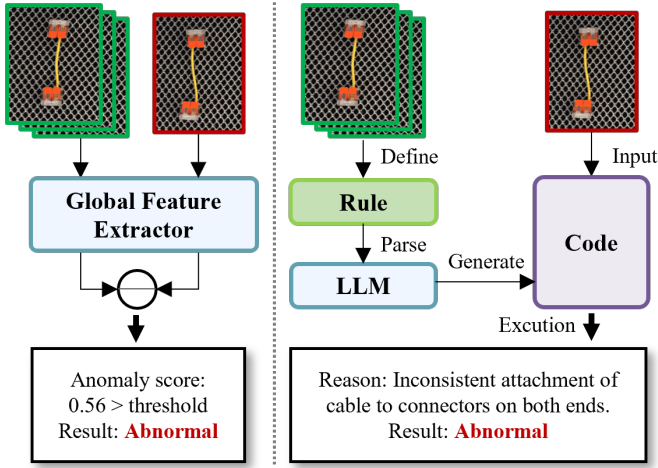


Fig. 2: Comparison between the existing method (**left**) and our LogiCode framework (**right**). The existing method do not accurately capture the underlying logical relationships and lack explanation for logical anomalies. In contrast, the proposed method starts from the perspective of logical constraints and gives the result with reason.

and analyzing these logical anomalies. Unlike traditional methods, LLMs can interpret and reason about high-level semantic logic, providing an innovative approach to anomaly detection.

Hence, this study proposes a LogiCode framework, utilizing the advanced capabilities of LLMs, representing a paradigm shift in logical anomaly detection. LogiCode, utilizing expert knowledge, extracts logical relationships from normal images and defines them as a set of logical rules. These rules are then parsed by the LLM, which automatically generates executable codes to examine the consistency between testing images and normal rules. Furthermore, the framework is designed to provide reasons for these anomalies, enhancing the interpretability of its findings, as shown in Fig. 2.

In this framework, the LLM's process of analyzing visual information and detecting logical anomalies is akin to the interplay between human perception and cognitive reasoning. LogiCode embodies this synergy, offering a more natural and intuitive approach to anomaly detection that mirrors human expertise. The framework transcends the capabilities of traditional models by providing enhanced interpretability, direct logic interpretation, and adaptability to the complex nuances of industrial quality control.

The key contributions of this study include:

- It proposes a new approach in logical anomaly detection with LogiCode, a framework that leverages the logical reasoning capabilities of LLMs to comprehend and interpret logical relationships. By generating executable Python codes, LogiCode emulates human-like reasoning, enabling precise detection and explanation of logical anomalies in industrial scenarios. This innovative approach marks a significant advancement in the field of logical anomaly detection.
- It introduces LogiBench, a benchmarking tool that integrates LLM automatic evaluation with expert manual analyses to evaluate the efficacy of LogiCode. LogiBench serves as a comprehensive measure for assessing the

accuracy and effectiveness of logical anomaly detection systems.

- It also contributes to the field by introducing the LOCO-Annotations dataset. It is meticulously designed to deeply analyze logical anomaly detection, encompassing a wide range of scenarios and providing a rich resource for both automatic and manual evaluations. To facilitate further research in logical anomaly detection, we have publicly released the LOCO-Annotations dataset at <https://github.com/22strongestme/LOCO-Annotations>.

Through these innovations, LogiCode not only addresses the existing gaps in logical anomaly detection but also enhances interpretability and adaptability in this field. Moreover, it sets a new benchmark in the application of LLMs for logical anomaly detection as a precedent for future exploration.

The reminder of this paper is organized as follows. Section I provides a comprehensive overview of the proposed framework and its position within the context of current research. Section II reviews some related work and establishes the foundation for the LogiCode framework. Section III details the methodological approach of LogiCode, followed by Section IV, which presents an in-depth analysis of experimental results and performance evaluations. Finally, Section V discusses the implications of our findings and explores potential directions for future research in logical anomaly detection using LLMs.

II. RELATED WORKS

This section delves into advancements in anomaly detection, including structural and logical approaches, as well as studies utilizing LLMs for automation tasks.

A. Structural Anomaly Detection

Existing methods that excel in detecting low-level structural anomalies can be categorized into two main types: reconstruction-based and feature-embedding similarity-based approaches.

Reconstruction-based methods, such as Autoencoders [14] (AEs) and Generative Adversarial Networks [15] (GANs), attempt to reconstruct input images through a lower-dimensional bottleneck. In this context, Yang et al. [16] proposed a multi-scale feature-clustering-based fully convolutional autoencoder that significantly improves the speed and accuracy of visual inspection for texture surface defects. Further advancing this category, innovations like RIAD [17] reformulates reconstruction as an image inpainting problem. While these methods are efficient in capturing image contexts [18], they often yield blurry reconstructions, leading to increased false positives.

On the other hand, feature-embedding based methods, such as RD4AD [19], Padim [20], PatchSVDD [21], SPADE [22] and PatchCore [23], utilize deep neural network-extracted vectors representing the entire image. Cao et al. [24] explored informative knowledge distillation within image anomaly segmentation to enhance model learning from complex data distributions. Moreover, Jiang et al. [25] introduced a masked reverse knowledge distillation approach that leverages both global and local information for enhancing image anomaly detection. These methods, relying on the distance between the

embedding vectors of test images and normal reference vectors from the training dataset [26], [27], demonstrate superior performance compared to reconstruction-based approaches [28].

Both reconstruction-based and feature-embedding based approaches have shown great result, on datasets for structural anomalies like MVTec AD [4]. However, these methods, constrained by their limited receptive fields, struggle to detect anomalies beyond receptive fields and fail to distinguish violations of logical constraints. This is particularly true in LOCO datasets, where the detection of logical anomalies is crucial. This highlights a significant gap in current anomaly detection methodologies, underlining the need for more adaptable and comprehensive systems that can effectively detect logical anomalies.

B. Logical Anomaly Detection

In addressing high-level semantic logic anomalies, several methods have emerged in recent research. MVTec LOCO [5], as a milestone, is specialized for studying and resolving advanced semantic logic anomalies. Alongside, GCAD [5] was proposed to solve logical anomaly detection. GCAD includes a local network branch and a global network branch. The local branch is designed to detect novel local structures in images, while the global branch learns a global consistency representation through a bottleneck structure, targeting violations in long-distance dependencies. This structure allows for the detection of both structural and logical anomalies. Subsequent methods, focusing on logic detection, often continue to employ the strategy of GCAD's global network branch, which is based on vision-based global-local correspondences. For example, EfficientAD [8] utilizes an autoencoder-student network pair to detect anomalies that violate global semantic constraints. Similarly, GLCF [7] achieves this through a combination of local and global network branches; the local branch focuses on structural anomaly detection, while the global branch captures logical anomalies through semantic bottlenecks. However, global feature extraction may not always discern subtle logical anomalies, like correct component quantities arranged in the wrong order, or components of the right type but incorrect specifications. Therefore, employing the logical reasoning capabilities of LLMs offers a more universal approach for resolving logical anomalies.

C. LLMs in Automation Tasks

The advance of LLMs marks a significant milestone in the field of automation, especially in industrial applications. These models have transcended traditional boundaries, offering novel solutions in code generation [13], visual question answering (VQA) [12], [29], and anomaly detection [30].

ViperGPT [13], a frontrunner in this domain, exemplifies the integration of LLMs in VQA, using code-generation to resolve visual queries more effectively than traditional methods. This represents a significant advancement over previous code generation techniques, providing more context-aware and adaptable solutions. The industrial application of LLMs extends to defect detection, anomaly detection, and quality

control. Zhang et al. [31] demonstrate its capability in zero-shot anomaly detection, applying it to complex VQA tasks. Wang et al. [32] introduces Industrial-GPT, tailored for intelligent manufacturing and excelling in tasks like fault diagnosis. Furthermore, Cao et al. [33] illustrates LLM's effectiveness for anomaly detection in multi-modal domains, while also acknowledging some limitations in intricate scenarios. AnomalyGPT [30] also emerges as an innovative industrial anomaly detection model, aligning images with textual descriptions to improve anomaly detection.

By leveraging the advancements in LLMs for logical reasoning and code generation, and addressing the limitations of existing logical anomaly detection methods, LogiCode represents a significant evolution in tackling high-level semantic logic anomalies in industrial settings.

III. PROPOSED LOGICODE FRAMEWORK

The LogiCode framework is an LLM-empowered framework developed for identifying logical anomalies in images. It is systematically introduced in the following subsections.

A. Framework Overview

The LogiCode framework is structured into three interconnected main modules—Code Prompt, Code Generation, and Code Execution, as shown in Fig. 3. Drawing inspiration from human cognitive processes, the framework adeptly combines observation and logical reasoning in its modules. The Code Prompt module initiates the process of identifying problems and formulating logical rules, akin to human problem recognition and solution crafting. Code Generation, powered by an LLM, processes these rules to create logical decision codes and selects corresponding visual APIs, reflecting human cognitive synthesis for decision-making. Code Execution module integrates logical evaluation with visual parsing, analogous to human reasoning and information interpretation, focusing on essential visual details. This module's output offers a detailed analysis, identifying anomalies with rule-based explanations, showcasing the framework's human-like problem-solving approach. The collective functionality of these modules highlights how the LogiCode framework excels in logical anomaly detection and interpretability, both essential for industrial applications.

B. Code Prompt

Code Prompt serves as the initial stage of the logical anomaly detection process. This module combines the functionalities of "Request Prompt" and "Logical-Rule Definition" to define the analyses task and establish the corresponding logical rules:

Request Prompt: Within "Code Prompt", the "Request Prompt" sub-module is meticulously designed to formulate the user's task definition into a structured prompt for the utilized LLM. This module involves several components, each playing a critical role in transforming complex tasks into structured LLM prompts:

Task Interpretation: Request Prompt starts with a clear interpretation of the task, as in "Create a Python function utilizing

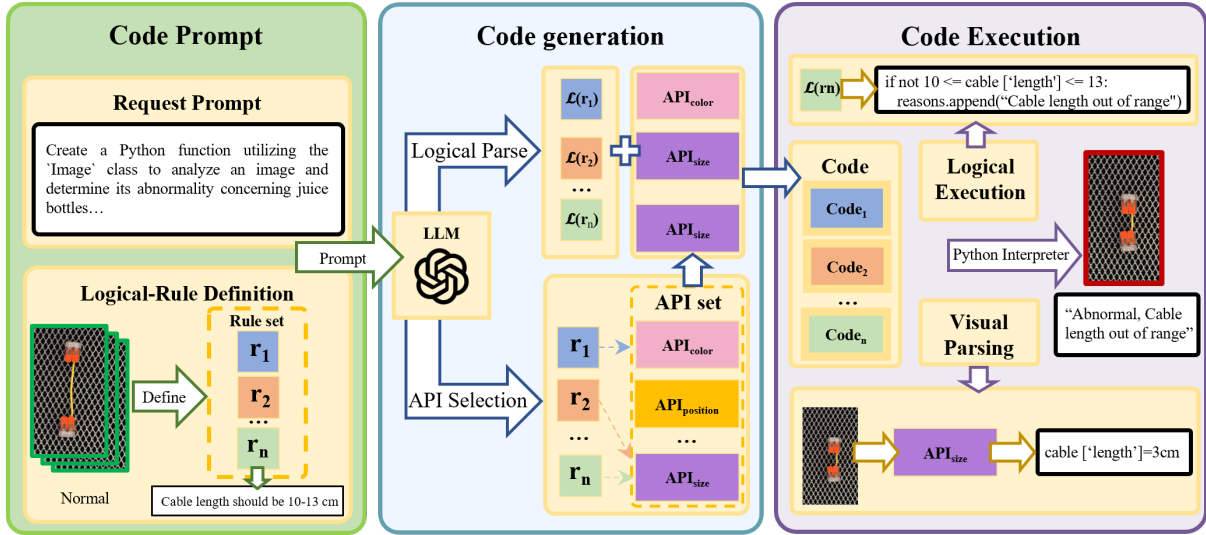


Fig. 3: **Overview of LogiCode framework** LogiCode is an LLM-empowered logical anomaly detection framework. It comprises three main modules: *Code Prompt*, *Code Generation*, and *Code Execution*. a) *Code Prompt* module formulates user-defined tasks and logical rules by analyzing examples of normal and abnormal images. b) *Code Generation* module utilizes an LLM to parse these rules into executable Python codes, selecting appropriate APIs for image analyses. c) *Code Execution* module applies logical and visual parsing to detect and report anomalies. This comprehensive process not only detects anomalies but also provides rule-based explanations for them, mimicking human problem-solving and decision-making abilities in industrial settings.

the Image class to comprehensively analyze an image and determine its abnormality concerning its cable and connectors layout,” ensuring task needs are precisely articulated in LLM-compatible formats.

Function Structuring: Simultaneously, Request Prompt organizes the prompt to include all critical aspects of the function, such as inputs and expected outputs, ensuring that instructions like “the function should accept the image path and return two results: a boolean and a string listing all the reasons for abnormality,” are clearly communicated to the LLM.

Knowledge Integration: Domain-specific knowledge is then woven into the Request Prompt, guiding the system to “Employ basic Python features for logical operations and mathematical calculations,” thus grounding the task in the relevant technical context.

Prompt Engineering: Inspired by the latest research in prompt engineering [34], [35], [36], this component incorporates strategies like outlining sub-tasks and illustrating relationships, like “Initially, outline the sub-tasks required for the analyses. Illustrate the relationships between these sub-tasks through a detailed step-by-step breakdown.” This approach guides the LLM in understanding the structure and logic of the task, enhancing the accuracy and efficiency of the code generation process.

Logical-Rule Definition: Within “Code Prompt”, the “Logical-Rule Definition” is a critical sub-module that operates in conjunction with the “Request Prompt”. It is focused on defining logical rules with expert knowledge from observations of normal example images.

Each set of rules is scenario-specific and can be adapted to fit a wide range of contexts, making the framework versatile and applicable across different industries. The LogiCode framework, through this sub-module, showcases its adapt-

ability by allowing these rules to be modified or expanded based on the requirements of different industrial scenarios, thereby maintaining the relevance and efficacy of the anomaly detection process.

C. Code Generation

Following Code Prompt, Code Generation in the LogiCode framework serves as a crucial bridge between conceptual rules and executable codes. This module harnesses the power of the LLM to transform structured request prompts and logical rules into Python codes. The process involves several key components:

Logical Parsing: Once the LLM receives the structured prompts, it employs its extensive knowledge base to parse complex logical conditions into executable Python codes. For example, a rule concerning the length of cable is interpreted by the LLM as:

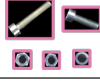
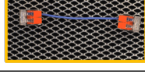
$$L(r_{\text{length}}) : \text{if } L_{\text{cable}} \notin [L_{\min}, L_{\max}] \quad (1)$$

L_{cable} is the measured cable length, and $[L_{\min}, L_{\max}]$ is the acceptable length range for the cable. Each rule r from the set R is parsed into a logical expression $L(r)$ that the Python interpreter can execute.

API Selection: The designed Image class provides a rich API set that the LLM selects from to match the logical rules with the functional capabilities needed for image analyses, as shown in Table I. This selection is crucial as it dictates the precision and efficiency of the subsequent image processing. The LLM matches each logical rule r from the set R with an API function $\mathcal{A}(r)$ from the Image class, ensuring that each aspect of the image analyses is addressed.

Each method $\mathcal{A}(r)$ in the API set is chosen for its ability to provide the necessary data for the logical rules to operate on.

TABLE I: Illustration for API set functions

API function	Explanation	Example
find():	It forms the initial step in the LogiCode framework, extracting object location information from the Loco-Annotations.	 → 
locate():	It pinpoints the position of each instance, crucial for understanding the spatial layout within the image.	 → (X_{orange}, Y_{orange}) $(X_{img_label}, Y_{img_label})$
count():	It provides the capability to enumerate instances that meet specific criteria, a vital step in quantifying anomalies within the image.	 → pushpin num : 17 cell num : 15
determin_color():	It applies color analysis to each instance, ensuring that color consistency is maintained across the image.	 → cable color : blue
size():	It assesses the dimensions of each instance and calculates size related attributes, informing subsequent tasks based on size discrepancies.	 → juice height: 13 cm

For example, the size() method, used in adherence to a rule regarding object size, is synthesized into the code as follows:

$$\mathcal{A}(r_{\text{length}}) : API_{\text{size}}(\text{image}) \rightarrow L_{\text{cable}} \quad (2)$$

This expression normalizes how the size() method is selected and utilized to extract the length of cable, denoted by L_{cable} , from the given image. It encapsulates the action of the LLM in choosing the size() method from the Image class when the rule r_{length} is concerned with verifying the length of the cable within acceptable limits.

Code Synthesis: After logical parsing and API selection, the LLM synthesizes the information into a cohesive block of Python code. This synthesis combines the logic and API calls into a sequence of executable instructions:

$$c = \bigcup_{r \in R} \{\mathcal{L}(r), \mathcal{A}(r)\} \quad (3)$$

The SynthesizeCode component illustrates the LLM's action in compiling the parsed rules and selected APIs into a final code segment ready for execution.

Code Generation thus serves as a crucial bridge, translating the structured input from the Code Prompt into Python codes that are ready for execution. It uses LLM's inferential power and the APIs' feature extraction capabilities to perform detailed image analyses and accurately identify logical anomalies.

D. Code Execution

Code Execution is the operational core of the LogiCode framework where the Python codes synthesized by Code Generation are brought to execution. This module interprets and runs the codes on the given image data, applying the logical rules and utilizing the APIs to perform a detailed analysis of the imagery. It consists of two critical sub-modules, each responsible for different aspects of the execution process:

Logical Execution: This sub-module is responsible for the logical assessment of each image based on the generated codes. It executes the conditional statements and loops derived from the logical parsing process, directly applying them to

the image data. For instance, the rule concerning the cable length is translated into an executable code that is conducted as depicted in the Code Execution section of Fig. 3. The logical execution is facilitated by the Python interpreter, which assesses the conditions set by each rule against the actual image data.

Visual Parsing: Integrated within Code Execution, visual parsing is crucial for the extraction of visual features from the image that are necessary for logical anomaly detection. This sub-module uses the selected APIs to determine the size, position, color, and other relevant attributes of the image components. The process of measuring the cable's length in the image, for instance, is exemplified in the orange section of the figure. The outcome of size is subsequently appraised by the predefined logical rules to ascertain its congruence with the acceptable size range.

Then the execution process integrates the outputs from logical execution and visual parsing, cross-referencing them to identify any anomalies. The integration ensures that all the visual features extracted are checked against the logical conditions, providing a comprehensive and accurate assessment.

The final output from Code Execution includes a detailed analysis report, highlighting any detected anomalies with their corresponding reasons. The report is designed to be both thorough and interpretable, allowing for quick identification of issues and facilitating decision-making processes.

By combining logical reasoning with visual data interpretation, Code Execution encapsulates the functionality required to identify logical anomalies accurately, reflecting a significant advancement in the application of LLMs to industrial anomaly detection.

IV. EVALUATIONS AND DISCUSSIONS

This section evaluates the proposed LogiCode framework's logical anomaly detection in industrial contexts using a specialized dataset, the LOCO-Annotations. It includes thorough benchmarking with LogiBench to test classification accuracy, code generation, and reasoning about anomalies. These tests aim to validate the LogiCode's capabilities for industrial

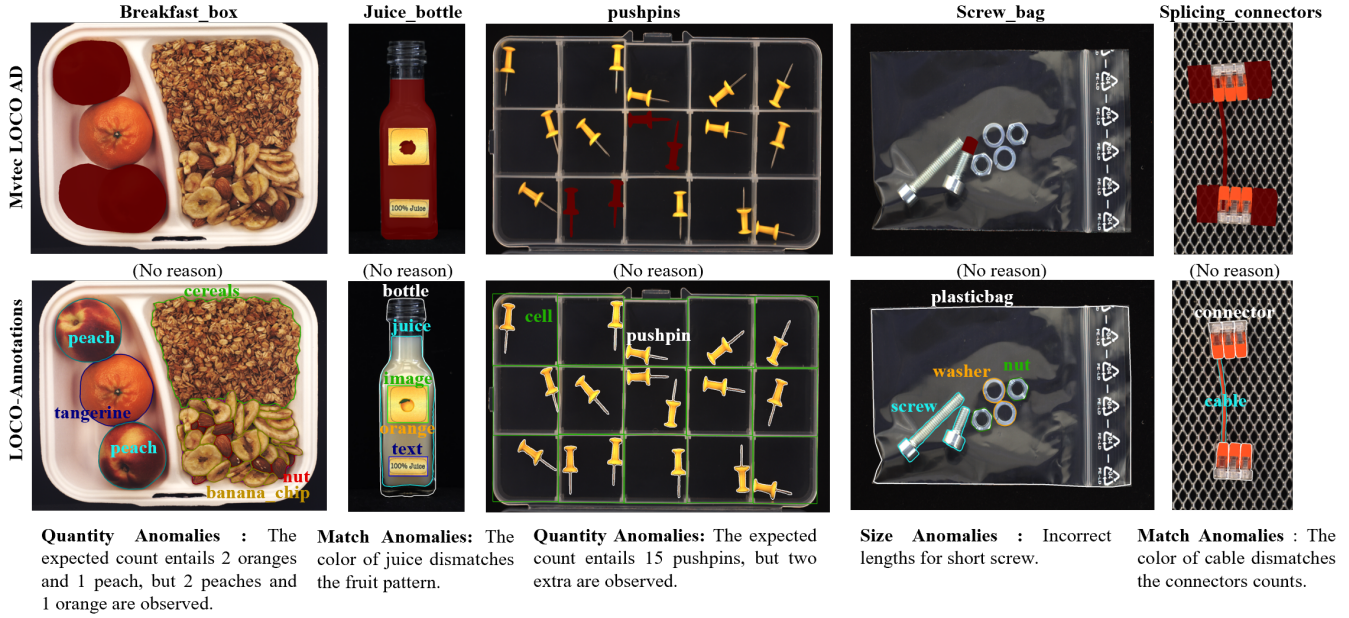


Fig. 4: The diagram contrasts the MVtec LOCO dataset (**Top**) with the LOCO-Annotations (**Bottom**). Unlike MVtec’s broad focus, the LOCO-Annotations dataset zeroes in on logical anomalies, offering an enriched dataset with detailed explanations for each anomaly. This provides LLMs with the context and specificity required for a nuanced understanding of anomalies, thereby facilitating a more sophisticated and targeted development in industrial quality control systems.

applications. Then the effectiveness and impact of LogiCode are discussed, highlighting its technological advancements and potential for application.

A. LOCO-Annotations Dataset

This subsection introduces the LOCO-Annotations dataset, a specialized extension of the MVtec LOCO dataset, aimed at filling a critical gap in the realm of logical anomaly detection in industrial scenarios. While the original MVtec LOCO dataset offers a robust foundation with its mix of structural and logical anomalies in industrial images, it primarily adheres to unsupervised anomaly detection, with a significant emphasis on detecting regions indicative of logical anomalies. Yet, the practical application of anomaly detection often necessitates an in-depth understanding of the underlying causes of these logical anomalies. In response to this, LOCO-Annotations shifts its focus from mere anomaly region detection to a thorough analysis of the underlying reasons for logical anomalies. This pivot aligns with the nuanced requirements of detecting and understanding logical anomalies, thereby better addressing the complex quality control demands in industrial environments.

LOCO-Annotations, comprising 2908 meticulously annotated images (1772 training and 1136 testing images, spanning various categories such as breakfast boxes, screw bags, pushpins, splicing connectors, and juice bottles.), diverges from MVtec LOCO’s approach by solely concentrating on logical anomalies. It categorizes the logical anomalies into four main types based on scenarios: Quantity Anomalies, Size Anomalies, Position Anomalies, and Matching Anomalies, as demonstrated in Fig. 5. This focus stems from a need to delve deeper into high-level semantic inconsistencies often overlooked in existing logical anomaly detection methods.

Unlike MVtec LOCO, which includes both logical and structural anomalies in its testing set, LOCO-Annotations dataset exclusively annotates logical anomalies in its testing subset. Each image in the LOCO-Annotations is accompanied by detailed JSON files providing pixel-level object segmentation and ground truth annotations formatted as “Anomaly Type: Specific Reason”, clarifying the reasoning behind each identified logical anomaly. Examples of MVtec LOCO and LOCO-Annotations are illustrated in Fig. 4. This level of detail is pivotal in training and evaluating logical anomaly detection methods. The unique contribution of LOCO-Annotations lies in its detailed focus on logical anomalies and their underlying reasons, providing essential data for testing and refining models like LogiCode and other LLM-driven methods. This approach aligns with the evolving needs of industrial quality control, where understanding the “why” behind logical anomalies is as crucial as detecting them. Additionally, its provision of data for decoupling individual objects within images further enhances the dataset’s utility, enabling a more detailed and nuanced analysis that is critical for understanding the complex dynamics of logical anomalies in industrial settings.

By offering a dataset that emphasizes logic and context, we aim to change the paradigm in anomaly detection research, focusing on logical semantic inconsistencies and their implications in industrial settings. The LOCO-Annotations not only fills the gap in industrial anomaly detection research but also paves the way for future advancements, setting new standards for the development of LLMs-based inspection systems.

B. LogiBench

LogiBench is meticulously designed for evaluating the LogiCode framework, with an emphasis on the thorough analyses

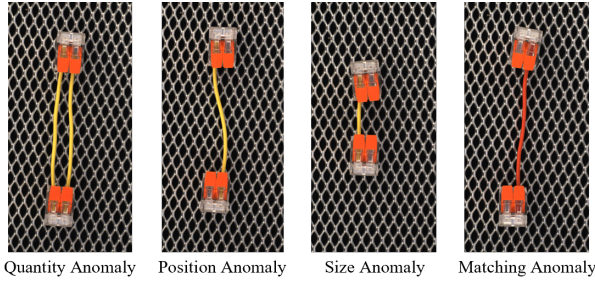


Fig. 5: **Quantity Anomaly:** The expected count for the cable is one, yet two is present. **Position Anomaly:** Cable should attach to the connectors with the same order defined by position; however, misplaced attachment is observed here. **Size Anomaly:** The cable length is anticipated to fall within a specific range; however, it is observed to deviate from the normal range. **Matching Anomaly:** The cable color is expected to match connector number; however, a mismatch is observed here.

of logical anomaly detection in industrial scenarios. The benchmark's construction is driven by the need to evaluate the framework's binary classification accuracy, code generation success rate, and reasoning accuracy:

Binary Classification Accuracy: This metric assesses the performance of the LLM-generated codes in correctly identifying the presence of logical anomalies in images.

It involves comparing the outcomes of the model with the ground truth (Normal/Abnormal) provided in the LOCO-Annotations to calculate crucial binary classification metrics like accuracy and recall rate. It is evaluated based on accuracy, precision, recall and F1-score.

Code Generation Success Rate: This metric measures the LLM's ability to generate executable Python codes based on the provided logical rules and request prompts.

This metric specifically focuses on comparing the explanations generated by LLMs with the standard truth reasons provided in the LOCO-Annotations, thereby determining the correctness of the LLM's reasoning against established benchmarks. It is crucial due to the potential variability in LLMs' ability to generate Python codes for logical anomaly detection.

The success rate is determined by analyzing whether the generated codes run without syntax errors and correctly implement visual APIs.

Reasoning Accuracy: This metric evaluates the precision of LLM-generated codes in explaining the reasons for logical anomalies. It is assessed to address the observed instances where LLMs correctly classify anomalies but may provide inaccurate explanations for their reasoning. The assessment is two-fold, detailed as follows:

Human Evaluation: An expert analysis to validate the consistency and correctness of the explanations provided by the LLM. It is necessary due to the complexity involved in discerning anomaly reasons, requiring expert knowledge to validate the consistency and correctness of the LLM's explanations.

LLM Automatic Evaluation [37], [38]: Utilizes a structured prompt to compare the output result against the ground truth reason, evaluating the LLM's accuracy in identifying logical

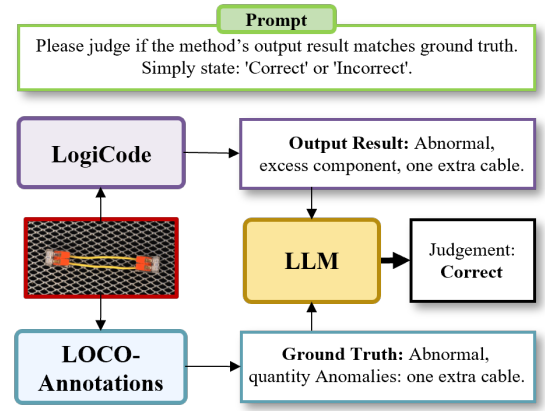


Fig. 6: Illustration for LLM Automatic Evaluation.

TABLE II: Performance of the proposed method

Category	Accuracy	Precision	Recall	F1 Score
Juice bottle	0.992	0.987	1.000	0.994
Breakfast box	0.989	0.976	1.000	0.988
Pushpins	0.990	0.976	1.000	0.988
Screw bag	0.981	0.965	1.000	0.982
Splicing connectors	0.989	0.978	1.000	0.989
Average	0.989	0.976	1.000	0.988

anomaly reasons, as shown in Fig. 6. It is included to reduce the workload of manual evaluation.

LogiBench sets a new precedent for evaluating logical anomaly detection algorithms, providing a detailed and multidimensional evaluation of the LogiCode framework's capabilities. Through this benchmarking process, we aim to demonstrate the advanced interpretability and efficiency of LogiCode in industrial scenarios, underlining its adaptability and robustness in a variety of industrial contexts. The thorough assessment provided by LogiBench also serves as a valuable resource for future enhancements and applications of LLMs in logical anomaly detection.

C. Evaluation Results and Analyses

The effectiveness of LogiCode, as assessed by the LogiBench benchmark, is presented in this subsection. Our comprehensive evaluation covers several key performance metrics, the results of which are summarized below.

At the outset, it is important to note that all experiments conducted in this study are based on GPT-4 [39]. Employing GPT-4 for this purpose reflects the framework's integration of state-of-the-art LLM capabilities to ensure the precision of its anomaly reasoning analysis. The outcomes of the various performance metrics assessed are presented below.

Binary Classification Accuracy: The accuracy, precision, recall, and F1-score of the framework in identifying logical anomalies are measured against the ground truth labels provided by the LOCO-Annotations.

The binary classification metrics are determined through comprehensive testing across all categories. The evaluation involved conducting five sets of experiments for each category, with the results averaged to obtain a reliable measure of

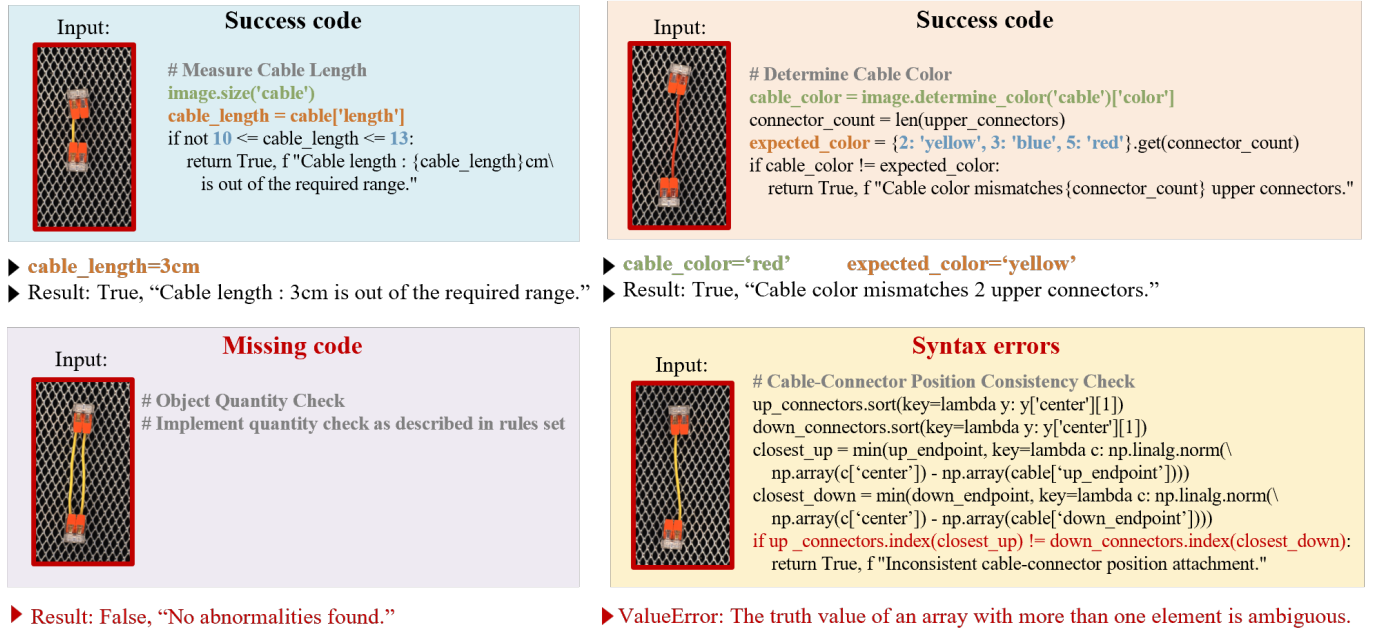


Fig. 7: Demonstration for success code, missing code and syntax error cases. The figure shows the core part of the generated code, the value of intermediate variables during the execution and the final output.

accuracy, precision, recall and F1-score. This approach ensures a thorough and unbiased assessment of the framework's ability to detect logical anomalies.

The results for the Binary Classification metrics are presented in Table II. It can be observed that the binary classification metrics for anomaly detection are remarkably high, suggesting that the proposed framework can effectively detect logical anomalies.

To comprehensively evaluate the efficacy of the proposed LogiCode framework in detecting logical anomalies, comparative analyses against current state-of-the-art (SOTA) methods have been conducted. As outlined in Table III, our assessment is meticulously focused on logical anomalies across various categories, contrasting our binary classification accuracy with that of established approaches. Unlike traditional methods, which predominantly rely on AUROC metrics for performance evaluation, the proposed framework necessitates a binary classification due to its inherent design to output discrete labels (0 for 'normal' and 1 for 'anomaly'). This differentiation necessitated the adoption of an alternative evaluation metric, focusing on maximized accuracy through variable threshold optimization.

The implementation of LogiCode, along with all competing methods, has been conducted in-house to ensure a fair and controlled comparison. Notably, the use of the LOCO-Annotations dataset allows us to leverage additional data, not merely for performance enhancement but as a means to showcase the inherent superiority of the proposed framework. The results, as depicted in Table III, affirm the exceptional performance of the proposed method, underscoring its potential to redefine standards in logical anomaly detection within industrial settings.

Code Generation Success Rate: This paper evaluated

the success rate of the framework in generating syntactically correct and executable Python codes.

To evaluate the success rate of code generation, code produced for each category are generated 20 times separately. The success rate is calculated based on the number of times the codes ran successfully and aligned with the logical anomaly rules without any Python syntax errors. This repeated execution method provides a robust assessment of the framework's consistency in generating executable Python codes.

Table IV presents the success rate of code generation using prompts in GPT-4, where "success" is attributed to instances where the codes' functionality aligns with the logical anomaly rules without Python syntax errors that could prevent execution. The other instances are categorized as "missing" or "error," as detailed in the Table IV. The examples for the generated codes are shown in Fig. 7.

It is observed that due to the current instability of LLMs, the code generation success rate presently hovers around 60%. However, it is expected that future iterations of LLM frameworks will significantly improve this metric.

Reasoning Accuracy: The framework's precision in articulating the reasons for anomalies has been tested through LLM automatic evaluation and human evaluation.

For reasoning accuracy, the evaluation focused on codes that successfully executed in each category. Five sets of such successful codes are analyzed for each category, with the results averaged to assess the framework's precision in articulating the reasons for anomalies.

Human Evaluation: The human evaluation involved employing three researchers with relevant backgrounds to assess the framework's reasoning accuracy. Each researcher independently evaluates the explanations provided by the framework, and their assessments are averaged to determine the overall

TABLE III: Accuracy comparison with the sota methods

Category/Methods	ST	PEFM	RD4AD	SPADE	GCLF	EfficientAD-S	EfficientAD-M	LogiCode
Juice bottle	0.741	0.731	0.767	0.806	0.935	0.970	0.976	0.992
Breakfast box	0.746	0.795	0.735	0.695	0.693	0.753	0.778	0.989
Pushpins	0.668	0.681	0.699	0.568	0.823	0.916	0.897	0.990
Screw bag	0.537	0.571	0.579	0.642	0.627	0.657	0.672	0.981
Splicing connectors	0.634	0.639	0.621	0.708	0.866	0.907	0.936	0.989
Average	0.665	0.683	0.680	0.684	0.789	0.841	0.852	0.989

TABLE IV: Code Generation Success Rate

Metric	Juice bottle	Breakfast box	Pushpins	Screw bag	Splicing connectors	Average
Success	0.350	0.650	0.800	0.600	0.550	0.590
Error	0.300	0.100	0.100	0.500	0.150	0.140
Missing	0.350	0.250	0.100	0.350	0.300	0.270

TABLE V: Human and LLM evaluation Reasoning Accuracy

Methods	Juice bottle	Breakfast box	Pushpins	Screw bag	Splicing connectors	Average
Human	0.901	0.974	0.976	0.988	0.976	0.963
LLM	0.869	0.928	0.932	0.936	0.954	0.924

accuracy.

LLM Evaluation: For the automatic evaluation of reasoning accuracy, GPT-4 is employed. The implementation details of this evaluation are provided in the LogiBench benchmark section.

As shown in Table V, the comparison between the automatic reasoning evaluation of LLM and the assessments by human experts indicates a close correspondence, suggesting the reliability of the LLM’s self-judgment in identifying the causes of logical anomalies. Moreover, the reasoning accuracy metric for the framework consistently exceeds 90%, affirming the framework’s efficacy.

These results provide crucial insights into the capabilities and potential areas of improvements for the LogiCode framework in the context of logical anomaly detection in industrial settings.

D. Discussions

The LogiCode framework introduces advancements and addresses limitations in existing LLMs in the context of industrial logical anomaly detection. This subsection explores the impact of LogiCode on this field, emphasizing both its advantages and the constraints of current LLMs.

Impact on Industrial Logical Anomaly Detection: LogiCode, integrating LLMs and specialized APIs, enhances the accuracy of detecting high-level semantic inconsistencies. This is particularly crucial in industrial settings where such anomalies can significantly affect product quality and safety. By providing more accurate and interpretable anomaly detection, LogiCode has the potential to transform quality control processes, reducing both time and costs associated with manual inspections.

Advantages and Limitations of LLM Methods: LLMs contribute a nuanced understanding of complex logical relationships, generating contextually relevant codes for anomaly

detection—a significant advancement from traditional rule-based systems. Despite their capabilities, LLMs may encounter challenges in highly specialized or niche industrial scenarios with limited training data. Furthermore, the black-box nature of these models can pose transparency and trustworthiness issues in critical applications.

Future Prospects and Recommendations: Future research could explore the integration of reinforcement learning and LLMs to autonomously summarize logical rules, reducing dependence on expert inputs and enhancing adaptability to new scenarios. The current reliance on detailed annotations for API functions such as “find” could evolve into more sophisticated zero-shot or one-shot methods, enabling generalized object segmentation without heavy reliance on pixel-level labels. This evolution would broaden the framework’s applicability across diverse industrial settings and decrease the need for extensive dataset preparations.

V. CONCLUSIONS

This paper introduces the LogiCode framework, a novel approach that leverages LLMs for industrial logical anomaly detection. LogiCode represents a significant transition from traditional methods, focusing on high-level semantic inconsistencies and offering a more nuanced understanding of logical anomalies. Its integration of LLMs for code generation and logical reasoning sets a new benchmark in the field, demonstrating remarkable adaptability and interpretability across various industrial scenarios.

Through rigorous evaluations with the LogiBench benchmark, LogiCode shows superior performance in binary classification accuracy, code generation success rate, and reasoning accuracy. These achievements underscore the framework’s efficiency and reliability in anomaly detection and its potential in revolutionizing quality control processes in industrial settings.

Looking forward, the potential of LLM-driven anomaly detection is vast. Future work will focus on further enhancing the models’ autonomy, allowing them to adapt seamlessly to a variety of scenarios with minimal human intervention. Additionally, efforts to improve the generalizability of the models to maintain consistent performance across different industries will be crucial.

In summary, LogiCode is more than just a solution to current challenges in logical anomaly detection, it is a foundation for a future where intelligent models are integral to industrial processes, offering robust, interpretable, and efficient solutions. The ongoing developments in LLMs and their applications in this field open up exciting possibilities for

industrial anomaly detection, leading the way toward more advanced, reliable, and automated quality control systems.

REFERENCES

- [1] Y. Cao, X. Xu, J. Zhang, Y. Cheng, X. Huang, G. Pang, and W. Shen. A survey on visual anomaly detection: Challenge, approach, and prospect. *arXiv preprint arXiv:2401.16402*, 2024.
- [2] X. Dong, C. J. Taylor, and T. F. Cootes. Defect classification and detection using a multitask deep one-class cnn. *IEEE Transactions on Automation Science and Engineering*, 19(3):1719–1730, 2021.
- [3] H. Yao, W. Luo, W. Yu, X. Zhang, Z. Qiang, D. Luo, and H. Shi. Dual-attention transformer and discriminative flow for industrial visual anomaly detection. *IEEE Transactions on Automation Science and Engineering*, 2023.
- [4] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger. Mvtec ad—a comprehensive real-world dataset for unsupervised anomaly detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 9592–9600, 2019.
- [5] P. Bergmann, K. Batzner, M. Fauser, D. Sattlegger, and C. Steger. Beyond dents and scratches: Logical constraints in unsupervised anomaly detection and localization. *International Journal of Computer Vision*, 130(4):947–969, 2022.
- [6] M. Yang, J. Liu, Z. Yang, and Z. Wu. Slsg: Industrial image anomaly detection by learning better feature embeddings and one-class classification. *arXiv preprint arXiv:2305.00398*, 2023.
- [7] H. Yao, W. Yu, W. Luo, Z. Qiang, D. Luo, and X. Zhang. Learning global-local correspondence with semantic bottleneck for logical anomaly detection. *IEEE Transactions on Circuits and Systems for Video Technology*, 2023.
- [8] K. Batzner, L. Heckler, and R. König. Efficientad: Accurate visual anomaly detection at millisecond-level latencies. 2024.
- [9] T. Gupta and A. Kembhavi. Visual programming: Compositional visual reasoning without training. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14953–14962, 2023.
- [10] X. Lai, Z. Tian, Y. Chen, Y. Li, Y. Yuan, S. Liu, and J. Jia. Lisa: Reasoning segmentation via large language model. *arXiv preprint arXiv:2308.00692*, 2023.
- [11] H. You, R. Sun, Z. Wang, L. Chen, G. Wang, H. Ayyubi, K. Chang, and S. Chang. Idealgpt: Iteratively decomposing vision and language reasoning via large language models. *arXiv preprint arXiv:2305.14985*, 2023.
- [12] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang. Hugginggpt: Solving ai tasks with chatgpt and its friends in huggingface. *arXiv preprint arXiv:2303.17580*, 2023.
- [13] D. Surís, S. Menon, and C. Vondrick. Vipergpt: Visual inference via python execution for reasoning. *arXiv preprint arXiv:2303.08128*, 2023.
- [14] G. Hinton and R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504–507, 2006.
- [15] S. Akcay, A. Atapour-Abarghouei, and T. Breckon. Ganomaly: Semi-supervised anomaly detection via adversarial training. pages 622–637, 2019.
- [16] H. Yang, Y. Chen, K. Song, and Z. Yin. Multiscale feature-clustering-based fully convolutional autoencoder for fast accurate visual inspection of texture surface defects. *IEEE Transactions on Automation Science and Engineering*, 16(3):1450–1467, 2019.
- [17] V. Zavrtanik, M. Kristan, and D. Skocaj. Reconstruction by inpainting for visual anomaly detection. *Pattern Recognition*, 112:107706, 2021.
- [18] P. Bergmann, S. Lowe, M. Fauser, D. Sattlegger, and C. Steger. Improving unsupervised defect segmentation by applying structural similarity to autoencoders. *arXiv preprint arXiv:1807.02011*, 2018.
- [19] H. Deng and X. Li. Anomaly detection via reverse distillation from one-class embedding. pages 9737–9746, 2022.
- [20] T. Defard, A. Setkov, A. Loesch, and R. Audigier. Padim: a patch distribution modeling framework for anomaly detection and localization. pages 475–489, 2021.
- [21] J. Yi and S. Yoon. Patch svdd: Patch-level svdd for anomaly detection and segmentation. 2020.
- [22] N. Cohen and Y. Hoshen. Sub-image anomaly detection with deep pyramid correspondences. *arXiv preprint arXiv:2005.02357*, 2020.
- [23] K. Roth, L. Pemula, J. Zepeda, B. Scholkopf, T. Brox, and P. Gehler. Towards total recall in industrial anomaly detection. pages 14318–14328, 2022.
- [24] Y. Cao, Q. Wan, W. Shen, and L. Gao. Informative knowledge distillation for image anomaly segmentation. *Knowledge-Based Systems*, 248:108846, 2022.
- [25] Y. Jiang, Y. Cao, and W. Shen. A masked reverse knowledge distillation method incorporating global and local information for image anomaly detection. *Knowledge-Based Systems*, 280:110982, 2023.
- [26] H. Yao, W. Yu, and X. Wang. A feature memory rearrangement network for visual inspection of textured surface defects toward edge intelligent manufacturing. *IEEE Transactions on Automation Science and Engineering*, 2022.
- [27] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger. Uninformed students: Student-teacher anomaly detection with discriminative latent embeddings. pages 4183–4192, 2020.
- [28] Y. Cao, X. Xu, Z. Liu, and W. Shen. Collaborative discrepancy optimization for reliable image anomaly localization. *IEEE Transactions on Industrial Informatics*, 2023.
- [29] J. Johnson, B. Hariharan, L. Van Der Maaten, L. Fei-Fei, C. L. Zitnick, and R. Girshick. Clevr: A diagnostic dataset for compositional language and elementary visual reasoning. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2901–2910, 2017.
- [30] Z. Gu, B. Zhu, G. Zhu, Y. Chen, M. Tang, and J. Wang. Anomalygpt: Detecting industrial anomalies using large vision-language models. *arXiv preprint arXiv:2308.15366*, 2023.
- [31] J. Zhang, X. Chen, Z. Xue, Y. Wang, C. Wang, and Y. Liu. Exploring grounding potential of vqa-oriented gpt-4v for zero-shot anomaly detection. *arXiv preprint arXiv:2311.02612*, 2023.
- [32] H. Wang, M. Liu, and W. Shen. Industrial-generative pre-trained transformer for intelligent manufacturing systems. *IET Collaborative Intelligent Manufacturing*, 5(2):e12078, 2023.
- [33] Y. Cao, X. Xu, C. Sun, X. Huang, and W. Shen. Towards generic anomaly detection and understanding: Large-scale visual-linguistic model (gpt-4v) takes the lead. *arXiv preprint arXiv:2311.02782*, 2023.
- [34] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. Elnashar, J. Spencer-Smith, and D. Schmidt. A prompt pattern catalog to enhance prompt engineering with chatgpt. *arXiv preprint arXiv:2302.11382*, 2023.
- [35] Q. Ye, M. Axmed, R. Pryzant, and F. Khani. Prompt engineering a prompt engineer. *arXiv preprint arXiv:2311.05661*, 2023.
- [36] T. Kojima, S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35:22199–22213, 2022.
- [37] X. Wang, Z. Wang, J. Liu, Y. Chen, L. Yuan, H. Peng, and H. Ji. Mint: Evaluating llms in multi-turn interaction with tools and language feedback. *arXiv preprint arXiv:2309.10691*, 2023.
- [38] Z. Bi, N. Zhang, Y. Xue, Y. Ou, D. Ji, G. Zheng, and H. Chen. Oceangpt: A large language model for ocean science tasks. *arXiv preprint arXiv:2310.02031*, 2023.
- [39] OpenAI. Gpt-4 technical report. arxiv 2303.08774. *View in Article*, 2:13, 2023.



Yiheng Zhang received the B.S. degree from the School of Mechanical Science and Engineering, Huazhong University of Science and Technology, Wuhan, China, in 2022, where he is currently pursuing the M.S. degree in mechanical engineering. His current research interests include industrial foundation models and their real-world applications, visual understanding, anomaly detection, and computer vision.



Yunkang Cao (Student Member, IEEE) received the B.S. degree in mechanical design, manufacturing and automation from Huazhong University of Science and Technology (HUST), Wuhan, China, in 2020, where he is currently pursuing the Ph.D. degree in mechanical engineering. His current research interests include industrial foundation models and their real-world applications, anomaly detection, and computer vision.



Xiaohao Xu received his B.S. degree in mechanical design, manufacturing and automation from Huazhong University of Science and Technology (HUST), Wuhan, China in 2022. He is currently pursuing the Ph.D. degree in robotics with the University of Michigan, Ann Arbor, MI, USA. His current research interests include the fundamental theory and real-world applications of robotics, computer vision, and video understanding.



Weiming Shen (Fellow, IEEE) received the B.E. and M.S. degrees in mechanical engineering from Northern Jiaotong University, Beijing, China, in 1983 and 1986, respectively, and the Ph.D. degree in system control from the University of Technology of Compiegne, Compiegne, France, in 1996. He is currently a Professor with the Huazhong University of Science and Technology (HUST), Wuhan, China, and an Adjunct Professor with the University of Western Ontario, London, ON, Canada. Before joining HUST in 2019, he was a Principal Research

Officer at the National Research Council Canada. He is a Fellow of Canadian Academy of Engineering and the Engineering Institute of Canada. His work has been cited more than 20000 times with an h-index of 70. He authored or coauthored several books and more than 560 articles in scientific journals and international conferences in related areas. His research interests include agent-based collaboration technologies and applications, collaborative intelligent manufacturing, the Internet of Things, and Big Data analytics